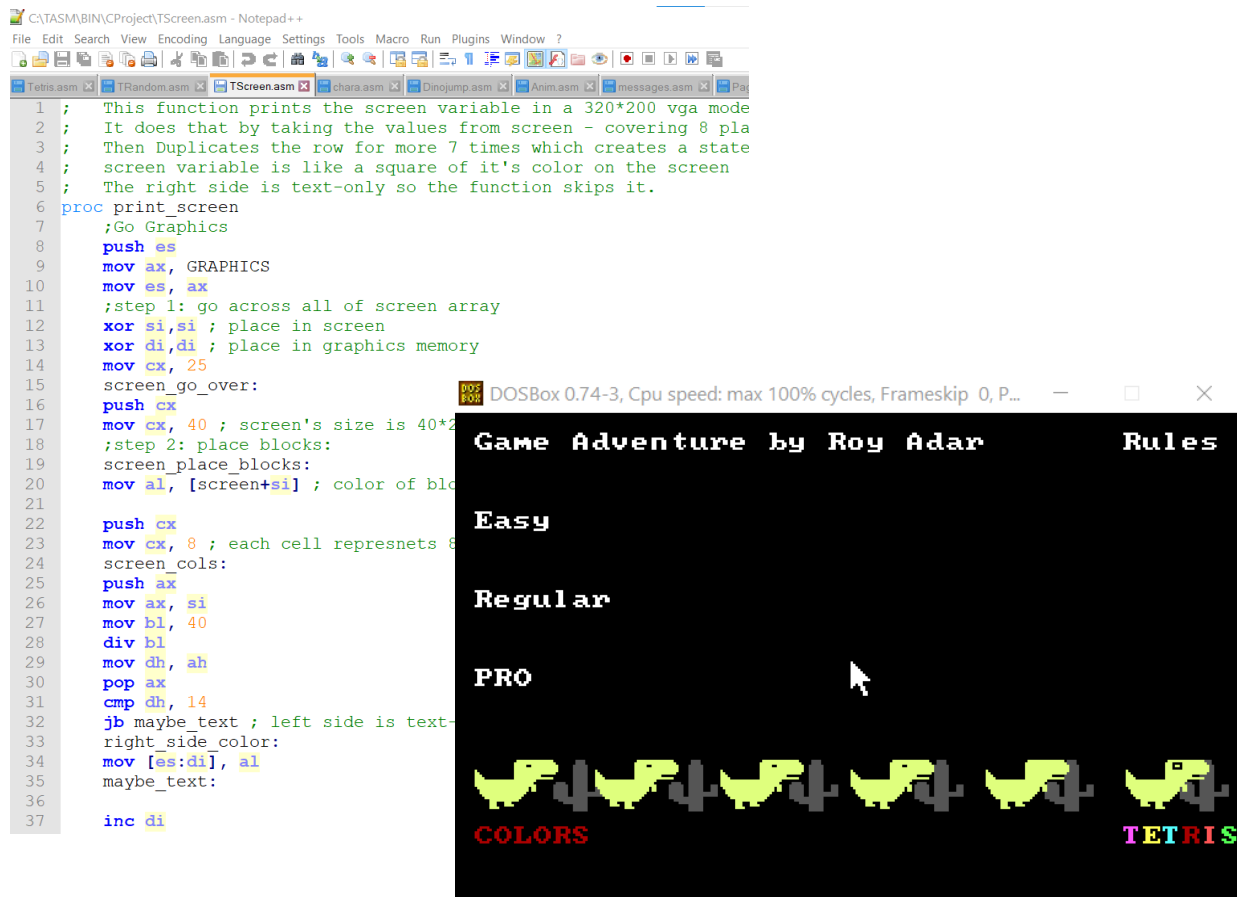


Tetris & Dino Game

Two Games – One Program

Created by Roy Adar



The code was written using notepad++ text editor

Runs on DOSBox, compiled with Borland Turbo Assembler

Features: Guaranteed “Fun”

תוכן עניינים

1	פתיחה
2	תוכן עניינים
3-6	על הקוד – בעיות ופתרונות
7-9	תיאור מסכים
10	אלגוריתמים מעניינים
11	רפלקציה
12-13	תרשימי זרימה
14-19	רשימת פונקציות
20	חלק קוד לדוגמה
21	רשימת קבצים ומשתנים לדוגמה
22	מקורות חיצוניים ומערכת הניקוד במשחקים

על הקוד

בעיות ופתרונות

משחק הדינוזאור :

```
proc draw_bird_up
  push es
  mov ax, GRAPHICS
  mov es, ax
  mov bp, [cac_place]
  mov al, [cactus_color]
```

בעיה מספר 1 : המשחק דורש מהירות רבה ועקב הדפסה של תמונות נוצרים ריצודים.

```
add bp,013
```

```
mov cx,002
```

```
call draw_line
```

```
add bp,317
```

```
mov cx,005
```

```
call draw_line
```

```
add bp,315
```

```
mov cx,007
```

```
call draw_line
```

פתרון : כאשר אנחנו מדפיסים מתמונה, אנחנו צריכים גם לעבור על התמונה וגם להעביר צבעים לזיכרון הגרפי, תוך כדי גם במקרים מסוימים צריך להתעלם מצבעי הרקע. לכן, יצרתי תוכנה פשוטה שמקבלת תמונה וממירה אותה לקוד שלם המצייר את הדמות (הקוד נשמר בקובץ טקסט) $\leq$ כך חוסכים איטרציות של מעבר על התמונה, ומעבר על צבעי רקע.

בקוד עצמו השתמשתי רק בקודים מציירי תמונה שיצרתי קודם בתוכנה. כיוון שהם ארוכים מאוד הם שמורים בקובץ משלהם על מנת לשמור על סדר.

*יתרון נוסף של השיטה – שינוי צבעים יותר בקלות.

*חיסרון – הקוד ארוך יותר ולכן הוא תופס יותר מקום בזיכרון.

בעיה מספר 2 : ריבוי תזוזות של דמויות שונות בו-זמנית.

פתרון : שימוש בפונקציית הזזת הקטוסים כפונקציית דיליי. כלומר $\leq$ נבצע את כל מה שנרצה לעשות, נקרא לפונקציית הזזת הקטוסים וכיוון שהקטוסים תמיד זזים לאורך המשחק נטמיע בהם את הדיליי שהתוכנה זקוקה לו.

בעיה מספר 3 : יצירת תנועה מתגברת (תאוצה).

פתרון : כיוון שממילא צריך להשתמש בדילי המינימלי האפשרי לתוכנה כדי שתרוץ בקצב סביר, נעלה את כמות הפיקסלים שהקטוסים עוברים בכל פעם, כל כמות מסוימת של נקודות (עד הגעה למקסימום דילוג פיקסלים).

בעיה מספר 4 : אנימציה כל X פעמים.

פתרון : משתנים שישמרו את כמות הפעמים שרצו על פונקציית האנימציה ורק לאחר הגעה למספר X יבצעו את האנימציה ויתאפסו.

בעיה מספר 5 : יצירת הלילה.

פתרון : פתרון טיפש – כתיבת קוד ארוך ומייגע לכל מצב צבע...

הפתרון החכם -> שימוש בתכונות צבעים באסמבלר 8086, פלטת הצבעים הבסיסית (16 צבעים) בנויה כך שאם נוריד מהמספר 15 צבע X נקבל את הצבע הנגדי לו. למשל נגדיר :

$X = 15$ (White), so: $15 - x = 15 - 15 = 0 \Rightarrow \text{Black}$

וכך בנויה הפונקציה invert_colors.

יתרון נוסף : לא צריך לשמור את הצבע הקודם כי הוא נשאר נגדי לצבע החדש.

בעיה מספר 6 : ניתור פגיעות דינוזאור בקטוסים.

פתרון : יצירת משתנה אשר ישמש כ-FLAG, הוא יודלק כאשר הפונקציה אשר מציירת בזיכרון הגרפי תנסה לצבוע פיקסל בצבע דינוזאור בצבע קטוס או להפך. לאחר מכן פעולה משלימה תוריד לדינוזאור חיים אם הדגל דלוק, ותבדוק אם הוא מת.



משחק Tetris :

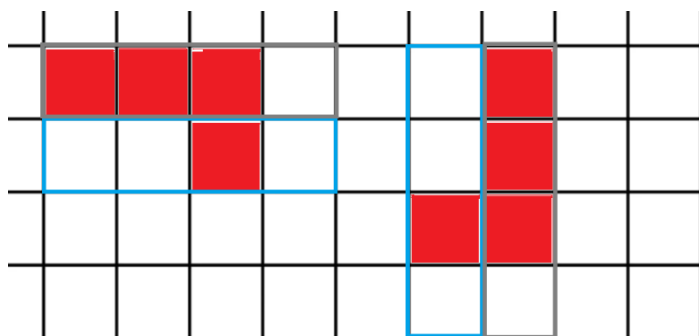
בעיה מספר 1 : יצירת הצורות ואנימציה איכותית, יעילה ומסודרת שלהן.

פתרון : יצרתי משתנה מסך, הוא בגודל 40 על 25, משתנה המסך מתאר מסך טטריס פשוט, כיוון שבטטריס כל דמות מורכבת מריבועים בעלי גודל שווה, כך גם במשתנה המסך, כל תא בו מתאר ריבוע 8 על 8 במסך האמיתי. יחד עם פעולות מיוחדות שהכנתי להעברת זיכרון ממשתנה המסך, למסך המשחק, ניתן פשוט להעתיק את הדמויות למשתנה המסך במקומן ואז להעביר את זה למסך המשחק האמיתי. הדבר מקל מאוד מבחינת ההסתכלות הכללית על התוכנה, נוחות אלגוריתמית, עקיבה אחר המתרחש על המסך (מיקום הצורות, המסגרת, שורות מושלמות וכו') בקלות רבה יותר, כך גם יותר קל לבדוק אם אפשר להזיז צורה או לא, אם אפשר לסובב צורה או לא וכו'. לסיכום, חילוק המסך לריבועים תורם רבות לפישוט משחק הטטריס ומקל בהמשך התכנות. בנוסף, השתמשתי במשתנה [block] בשביל לשמור את הכתובת בזיכרון של הצורה שכרגע נופלת וכך חסכתי הרבה אי-נוחות.

בעיה מספר 2 : סיבוב הצורות.

פתרון : פתרון טיפש – הגדרת צורות מסובבות בזיכרון – קוד מסורבל

האלגוריתם שהמצאתי לפתרון הבעיה – העברת השורות לעמודות בדמות, מהשורה התחתונה ביותר ומהעמודה הראשונה.



בעיה מספר 3 : להראות את הבלוק, הצורה שהבאה לרדת.

פתרון : ייצור מספר רנדומלי כבר בפתיחת התוכנה ולאחר מכן להמשיך לייצר מספר רנדומלי חדש כל פעם שבלוק נופל כך שהמספר שיצרנו לפניך ישמש לקביעת הבלוק במסך וזה שיצרנו עכשיו יהיה הבלוק הבא.

בעיה מספר 4 : בדיקת גבולות.

פתרון : פונקציית copy_try ומשתנה possible. כאשר ארצה לדעת אם אפשרי להעביר את הצורה למקום מסוים (הן בגלל צורה אחרת והן בגלל גבול המסגרת) אשתמש בפונקציה הנ"ל ואבדוק מה קרה למשתנה possible אם ערכו נשאר 0, זאת תנועה אפשרית, אחרת אם ערכו 1, הוא לא יכול להמשיך בדרך הזאת ולפי זה תמשיך התוכנית.

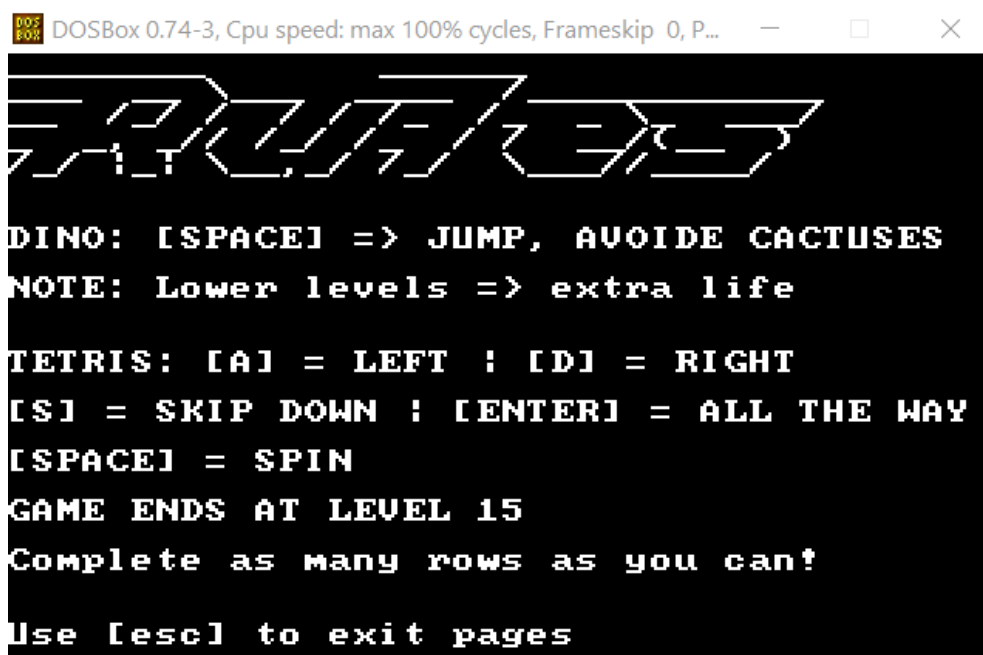


תיאור מסכים:

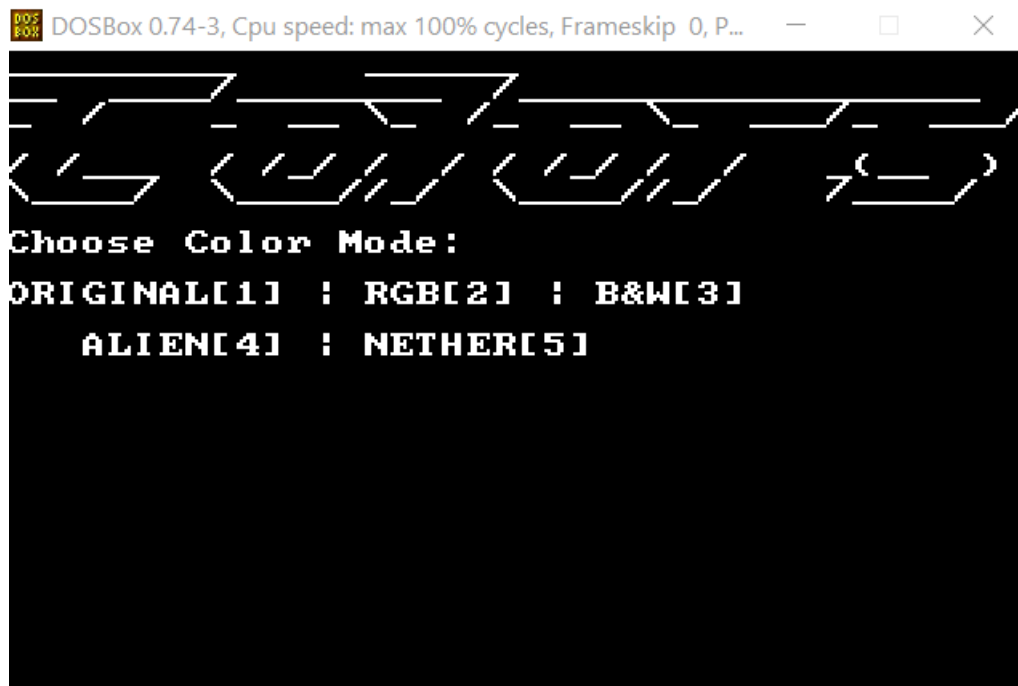
1. מסך כניסה, תפריט - מתופעל ע"י העכבר:



2. מסך החוקים (לאחר לחיצה על תווית Rules) - [esc] דרוש בשביל לצאת:



3. מסך הצבעים (משפיע רק על משחק הדינוזאור) כדי לצאת [esc], בחירת סוג צבעים עם הקלקה על מספר מתאים:



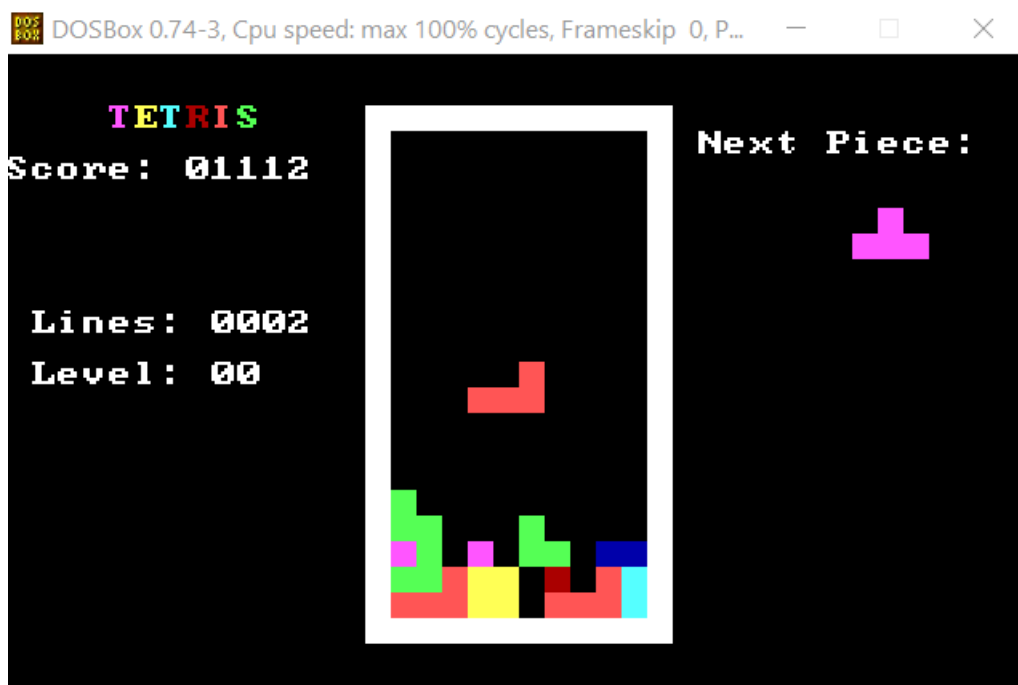
4. משחק הדינוזאור במצב צבעים NETHER, הקלקה על מקש רווח <= קפיצה (מטרת המשחק להתחמק מקטוסים רנדומליים, שבאים בזמן רנדומלי (אני מתייחס גם לציפורים כאובייקט מסוג קטוס)) :



*תכונה נחמדה: צבע המשחק בא לידי ביטוי במסך התפריט, לדוגמה - מצב NETHER:

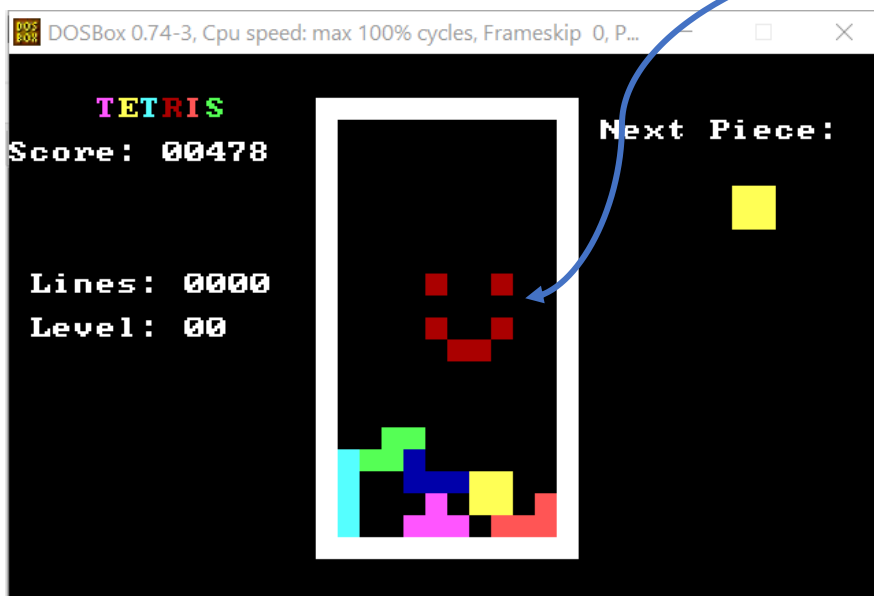


5. מסך משחק Tetris (המקשים כבר מופיעים בעמוד החוקים, מטרת המשחק להשלים כמה שיותר שורות ולהשיג כמה שיותר נקודות כך):



אלגוריתמים שראוי להזכיר (ולא הוזכרו לעיל):

1. אלגוריתם התיק הרנדומלי של Tetris : בגרסה שאני בחרתי לעשות כמו בהרבה גרסות אחרות התיק מורכב מכל 7 הצורות. איך זה עובד ? ככה, כמו באלגוריתם טיפוסי של ייצור מספרים "רנדומליים", האלגוריתם משתמש בשעון המערכת, ובבתים רנדומליים מהזיכרון, מבצע פעולות לוגיות ומחזיר מספר. אבל באלגוריתם הזה נוסף לנו התיק – התיק מאותחל עם כל 7 הצורות של המשחק. כל פעם המספר הרנדומלי מפנה אותנו לצורה בתיק, אנחנו מוציאים אותה והיא הצורה שמופיעה על המסך. אבל היא נשארת בחוץ, ולאחר מכן היא לא תוכל לצאת שוב עד שהתיק יתרוקן מכל 7 הצורות ונצטרך למלא אותו מחדש. הדבר מעניק לנו 7! שילובים שונים של צורות, כלומר 5040 רצפים אפשריים.
2. אלגוריתם הניסיון : לדעתי השימוש בפונקציה אשר מנסה לבצע את הפעולה ומחזירה האם היא אפשרית עזר מאוד ביעילות הקוד, ובפשטותו, כך הצלחתי לשמור על קוד קריא ונעים.
3. מודולריות בדמויות, הן במשחק הדינוזאור (שליטה מלאה על צבעים וכו') והן במשחק ה-Tetris, ניתן ליצור דמות חדשה רק על ידי שינוי הערכים אשר מייצגים דמות בזיכרון.



*אלגוריתמים מעניינים נוספים שכבר הופיעו : סיבוב, מסך מחולק לחלקים, כתיבת קוד ישיר לציור הדמות, היפוך צבעים, שימוש מורחב במשתנים – "דגלים".

רפלקציה:

במהלך כתיבת הפרויקט נתקלתי בבאגים ובעיות, חלקן היו בגלל טעויות סינטקס, אך אלו הן הקלות מבניהן, לא אשכח כמה זמן נתקעתי על זה שהיה חסר לי ret בסופה של פונקציה, או שדילגתי על pop וכו'. זה היה במשחק של הדינוזאור, שעשיתי קודם. הוא לימד אותי על כל הבעיות האלה ולכן הבחנתי באלה מיד במשחק ה-Tetris, שהכנתי לאחר מכן, וגם במשחק של הדינוזאור לאחר שלמדתי על הנקודות האלה, התמדתי בבדיקתן ואלה לא חזרו על עצמן. מה שכן, לא הכול היה מושלם, ולפעמים היו לי גם בעיות אלגוריתמיות, שאותן פתרתי בקריאה של הקוד מחדש. בנוסף, כמה פעמים עשיתי דיבאגינג בשיטה שעזרה לי מאוד - עם רשימת פקודה של הדפסה של מחרוזת בחלק מסוים של הקוד, רציתי לראות אם נכנסים לחלק הזה, כמה פעמים, מתי וכו' כלומר למצוא היכן הבעיה, ומהר מבלי להתעכב במעבר על הקוד כולו.

לסיכום, נהניתי מאוד מכתובת הקוד ובמשחק בתוצרים, אני מרגיש שלמדתי הרבה במהלך הפרויקט על אסמבלי ובכללי על דרכי חשיבה, הלמידה לתכנת באסמבלי פתחה לי כיווני מחשבה חדשים ותרמה לי מאוד. אסגור את הנושא בכך שאגיד שכמה שזה בסיסי העבודה עם שפת אסמבלי גרמה לחשוב על כל מה שקורה, במחשב, ובכל מקום אחר, כרצף אחד ענקי של כן ולא (כן, לא, כן, לא, לא, כן...) כלומר, בשפה טיפה אחרת, 0-1.

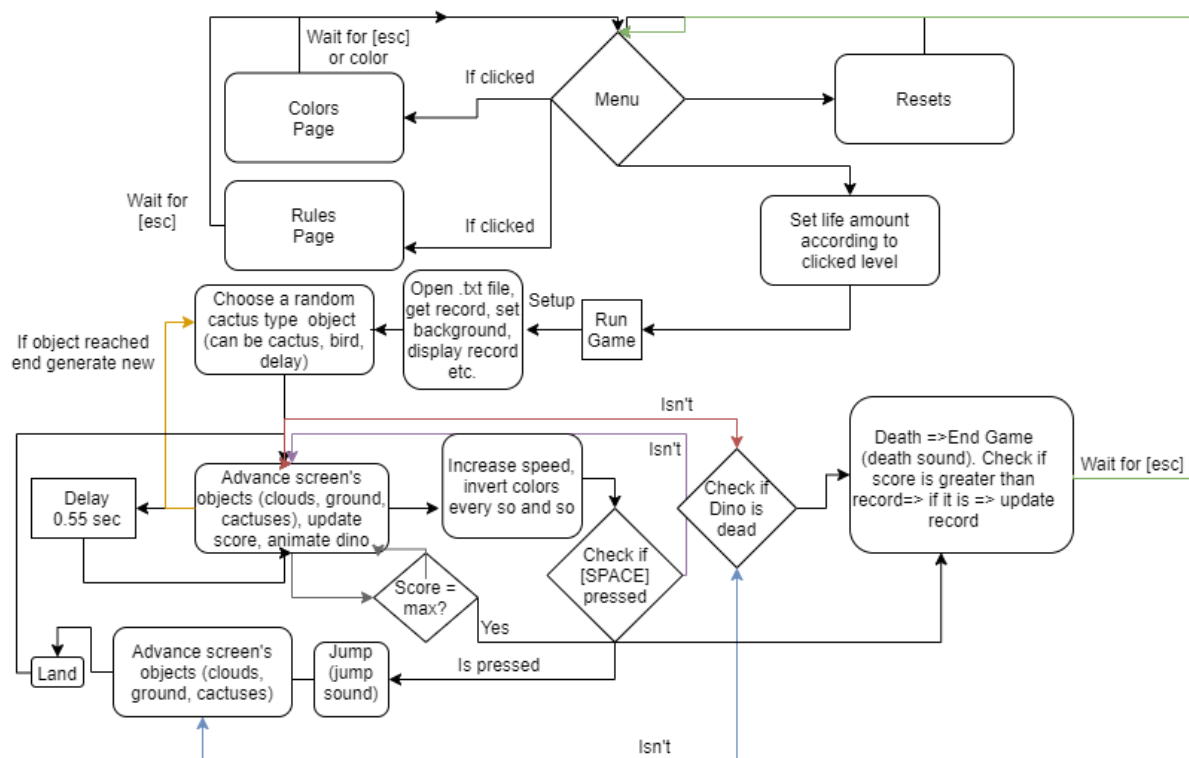


תרשימי זרימה כלליים, בסיסיים של שני המשחקים

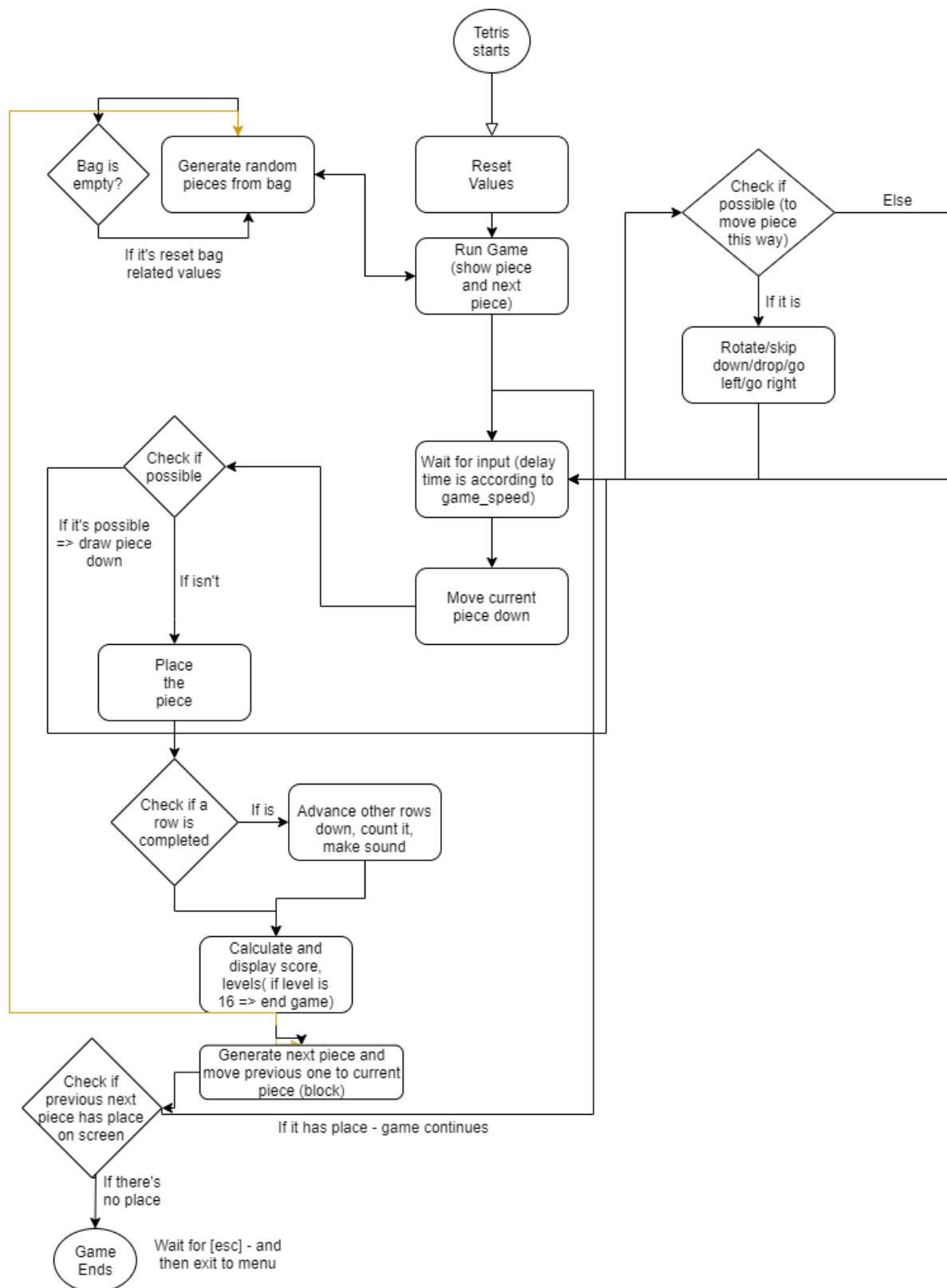
(התפריט בחלק של הדינוזאור)

התרשימים רק ממחישים את דרך הפעולה של המשחקים, אך בפועל מדובר בפעולות הרבה יותר מורכבות, ארוכות, הבלוקים בתרשימים רק ממחישים את מה שפונקציות וקטעי קוד שונים עושים במהלך המשחק על מנת שיפעל בצורה הרצויה ואת הבדיקות שנעשות – את הרעיונות העיקריים – "פיזיקת" המשחק כביכול

משחק הדינוזאור



Tetris



Functions List – “Dino Game” and tools:

mouse_status	Checks mouse status with interrupt
simple_print	Prints '\$' terminated string, gets address at dx
reset_game	Resets some values of dinosaur game
reset_cac	Resets the cactus object
cactus_move	Moves current cactus object, identifies it and sends it to the right function
invert_colors	Inverts the colors of dino game
speed_up	Speeds up the game at certain scores, also calls invert colors
draw_ground_line	draws the “ground line” in dino game
game	Runs the whole game
Animations:	
refresh_eye	Creates eye animation every ¼ times
refresh_legs	Creates Legs animation
clear_refreshed	Clears dino’s legs/eye animation to perform a jump
dino_jump	Creates a jump while cactuses keep move
dino_land	Lands the dino after the jump (bounded functions)
com_cac	Moves the common cactus object
cac_3	Moves the two-cactus object
cac_2	Moves the three cactuses object
cac_1	Moves the small-basic cactus object
draw_dead_dino	Draws a dead dino animation
redraw_cloud	Redraws the three clouds using cloud_drawer
cloud_drawer	Draws a cloud animation
generate_bird	Decides on a bird object (LOW/MID/HIGH)

mov_bird	Moves the bird object (animates wings - fly)
NOTE: animation functions gets height, colors etc. That are stored in certain vars.	
Characters:	
*Characters full drawing codes were generated by a machine according to an image. So, they are directly drawn to screen.	
draw_dino	Draws dino
small_cactus_1	Draws a small cactus (ver1)
small_cactus_2	Draws a small cactus (ver2)
basic_cactus	Draws a basic cactus
draw_cloud	Draws a cloud
draw_bird_up	Draws a bird (wing is up)
draw_bird_down	Draws a bird (wing is down)
three_cacs	Uses small cactus functions to draw a 3 cactuses object
common_cac	Draws a basic cactus object
two_cac	Draws two basic cactuses object
small_basic_cacs	Draws a small cac and a basic cac object
NOTE: draw functions draw the character according to certain height and color.	
Pages:	
rules_page	Displays the rules page (waits for [esc])
colors_page	Displays the colors page (gets input/waits for [esc])
menu	Resets games, displays menu, (waits for input – click on next page)
Others:	
random	Generates random number, puts in var
draw_ground	Draws_ground according to ground array
point	Draws a point

adv_ground	Advances the ground array (puts first val at last)
moving_ground	Animates ground and advances it like cactuses.
num_to_text	Gets a numbers address, and a string (message), puts the number in the string.
advanced_printing	Prints a string from @data using int 10h, 13, Gets: color in bl, length in cx, address at bp.
hi	Displays score in game-screen
Record file:	
open_file	Opens the file
write_to_file	Writes to the file
close_file	Closes it after work is finished
read_file	Reads from file
display_record	Displays the record on game-screen (uses the above file management functions)
compare_run_times	Compares record and current score
celebrate	If score passed record → celebration message (record also updates)
draw_line	Draws line while checking if dino hit cactus. (length in cx, color at al, draws at [es:bp], es = 0a00h)
simple_draw_line	Draws a line, length in cx, color at al, draws at [es:bp]] (es = 0a00h).
delay	Creates a short delay
make_Sound	Gets note in bx, opens speaker and sends it to it.
stop_sound	Closes speaker
death	Creates dino death scenario, stops game, displays message, creates a sound, death animation, etc.

	Waits for [esc] to go back to menu.
check_dead	Checks if dino was hit, and if he is now dead
set_background	Sets background at [background_color]
reset_cursor	Resets cursor...

Tetris Functions:

gen_block	Generates a new block, puts [next_block] in [block], and calls gen_random
block_identify	Identifies block's place in memory according to a number between 0-6
tetris_game	Runs the whole Tetris game, until player wins/loses
copy_to_screen	Copies shape to screen variable, (can also delete one if [clear_shape] says so.
copy_try	Trys to copy a shape to screen, if it can be copied => [possible] = 0, else [possible] = 1
draw_next_piece	Draws the next piece at screen's right side
tetris	Starts the Tetris game
gen_random	Generates a new shape from bag, randomly, refills bag if there's a need. (Puts its address in [next_block]).
print_screen	Prints [screen] to screen, using graphics memory, treating every [screen] value as a 8x8 square. (doesn't print screen's left side because it is text-only)
print_inner_board	Does as print_screen but only to the inner board part of the [screen]
rotate_shape	Rotates the shape with a special rotating algorithm.
clear_help_arr	Clears help_arr (puts 0 at all places)
copy_to_help_shape	Copies help_arr to help_shape
wait_for_key	Creates a delay while checking for input
get_input	Gets input does as it says – if possible (uses copy_try)
line_completed	Checks if a line was completed it => advances the other lines, counts it, generates sound.
shape_clear	Clears shape from screen (used for animation)

calc_score	Used for calculating score
display_score	Displays (prints) score
won_tetris	Displays victory message if all levels were completed
reset_tetris	Resets the Tetris game (screen, speed, score etc.)

*A lot of functions get data from variables or registers, I didn't specify it at the tables above because it will just be a mess to write it because it happens a lot.

I will describe some functions from the Tetris game for instance:

```
proc copy_try

    mov di, BOARD_FALL_START

    add di, [shape_height]
    add di, [shape_x_place]
    .....
```

The function of the above gets the shape's height and X position from these variables (and shape's address at bx).

It also returns if it's possible to copy the shape to this place at the [possible] variable.

```
proc rotate_shape

    mov bx, [block]
    mov cx, SHAPE_SIZE
    .....
```

This function gets a pretty different type of data, the [block] variable stores the address in the memory of a certain shape. So this function gets it and then rotates the shape at this address with an helping array, and finally switches the address that the [block] variable points to, to the rotated shape one (only if that's the first rotation of a shape, otherwise the address will remain the same).

So to sum-up it gets: shape at [[block]], and returns a rotated shape at [[block]].

*The following "[[]]" symbols are supposed to explain that the shape is at the place in memory that [block] points to but are illegal memory reference.

```

proc gen_random
    ....
    mov ah, [bag+si]
    mov [bag_took+si],1
    inc [taken]

    cmp [taken], 7
    jne no_need_to_fill_bag
    xor di,di
    mov cx, 7
    refill_bag:
    mov [bag_took+di], 0
    inc di
    loop refill_bag
    mov [taken], 0
    no_need_to_fill_bag:
    mov [next_block], ah
    ....

```

This function gets data from the [bag] variable (array), [bag_took] variable (array), and the [taken] counter variable.

It returns the random chosen shape's ID (identifier) , at [next_block].

קוד לדוגמה – סיבוב מערך דו-ממדי (Tetris)

```
116 proc rotate_shape
117     mov bx, [block]
118     mov cx, SHAPE_SIZE
119     ; First part - find the beginning row of the shape:
120     mov si, 16 ;    last place is [bx+15], 16 -1 = 15
121     find_beginning:
122     dec si
123     mov al, [bx+si]
124     cmp al, 0
125     je find_beginning
126     ; Second part:
127     ; Found it, now calculate how long does the rotating loop needs to be
128     ; and where do we start from
129     mov ax, 15 ;    15 => last place in shape's array
130     sub ax, si
131     mov bl, SHAPE_SIZE
132     div bl
133     xor ah, ah
134     mov cx, SHAPE_SIZE
135     sub cx, ax
136     mul bl
137     mov si, 15 ;    15 => last place in shape's array
138     sub si, ax
139     add si, [block]
140     mov di, cx
141     ; Third Part - rotating a 4x4 array (90deg):
142     rotate_cols:
143     push cx
144     push di
145     sub di, cx
146
147     mov cx, SHAPE_SIZE
148     rotate_rows:
149     push si
150     sub si, cx
151     inc si
152     mov al, [si]
153     mov [help_arr+di], al
154     add di, SHAPE_SIZE
155     pop si
156     loop rotate_rows
157     sub si, SHAPE_SIZE
158     pop di
159     pop cx
160     loop rotate_cols
```

רשימת קבצים

Filename	Usage
Main.asm	Holds the data segment, the main Dino-game function, other functions, and starts the program (sets graphics and calls menu) - Compile and run this file
Anim.asm	Dino game's animations functions
Chara.asm	Characters draw functions, as described at the beginning of this book
Record.asm	Record, score related functions (.txt file management, number to text converter...)
Pages.asm	The "boring" pages setups, (menu, rules, colors)
Random.asm	Dino's random object generator
Tools.asm	Other tools, like drawing tools, set background, delay, mostly used for Dino game.
Messages.asm	In-game messages are kept in a different file, so the data segment will be clearer.
Tetris.asm	The main Tetris, file, connects, pieces, and contains copy functions
TScreen.asm	Other Tetris related functions, like print screen, etc.
TRandom.asm	Tetris random shape (from bag) generator

משתנים מעניינים לדוגמה

Name	Usage
shape1	4x4 array which represents a tetris shape.
shape_x_place, shape_height	Stores tetris shape's place (There are similar variables for the dino)
possible	FLAG – that says if it's possible to move a shape in the checked way.
dino_color	Stores dino's color (there are bunch of this for: background, cactuses etc.) These are used in draw functions
hit_Cactus	FLAG – to know if Dino was hit.
cactus_object_type	Current cactus type object on screen.
cactus_skip	The number of pixels that current moving cactus-object (and ground of course) will skip on when they move. Increases with time.

מקורות חיצוניים

ספר גבהים - אסמבלי

http://data.cyber.org.il/assembly/gvahim_assembly_book.pdf

Art of assembly

<http://www.ic.unicamp.br/~pannain/mc404/aulas/pdfs/Art%20Of%20Intel%20x86%20Assembly.pdf>

Borland Turbo Assembler

http://bitsavers.informatik.uni-stuttgart.de/pdf/borland/turbo_assembler/Turbo_Assembler_Version_5_Users_Guide.pdf

Scoring system

Dino - points

Cactus moves	2
Cactus generated	5
Easter-egg: clicking 'f'	15

Dino- life

easy	3
regular	2
pro	1
Easter-egg: clicking 'd'	=> death
Easter-egg: clicking 'r'	100

Tetris- points

For every drawn square	1
1 completed row	40*level
2 completed rows	100*level
3 completed rows	300*level
4 completed rows	1200*level

*After reaching 60000 points the score will reset, the level will go up and the speed will decrease till reaching maximum level (a win), and minimum speed.

